Thai Shop Text-to-SQL Agent — Proposal & Design Doc

Oct 23, 2026 · @Arnuphap Monkolsuktavi

Proposal (ย่อหน้าเดียวสำหรับ milestone 1)

โปรเจกต์นี้สร้าง agent ที่แปลงคำถามภาษาธรรมชาติเป็น SQL บนข้อมูลสังเคราะห์ของร้านค้าออนไลน์ไทย (8 ตาราง ประมาณ 10,000 คำสั่งซื้อ ปี 2024–2025 สร้างด้วย script ที่ fix seed และจงใจใส่ข้อมูลไม่สะอาด) สำหรับนักวิเคราะห์ธุรกิจหรือผู้จัดการร้านที่ต้องการคำตอบเชิงตัวเลขแต่เขียน SQL ไม่คล่อง ระบบใช้ RAG ดึงคำอธิบายตาราง/คอลัมน์และตัวอย่างคำถาม→SQL ที่คล้ายกันมาใส่ใน prompt แล้วให้ agent loop (function calling, หยุดเมื่อไม่มี tool call หรือครบ 5 รอบ) เรียก tool ค้น schema และรัน SQL ผ่านตัวกรอง SELECT-only บน SQLite แบบ read-only พร้อมแก้ SQL เองเมื่อเจอ error คำตอบส่งผ่าน FastAPI แบบ SSE streaming โดยแสดงผลลัพธ์ SQL ที่ใช้ และ citation ของตาราง/ตัวอย่างที่ดึงมา การประเมินใช้ชุดทดสอบที่เขียนเองประมาณ 30 คำถามพร้อม gold SQL วัด execution accuracy เทียบ baseline แบบไม่มี retrieval (ใส่ schema ทั้งหมด, zero-shot, ยิงครั้งเดียว) และวัด recall@k ของการดึง schema เพื่อเลือก top-k/min_score ด้วยตัวเลขก่อน–หลัง

ปัญหาและกลุ่มผู้ใช้

คำถามธุรกิจส่วนใหญ่ต้องตอบด้วยการ join หลายตาราง เช่น "หมวดสินค้าไหนมีคะแนนรีวิวเฉลี่ยต่ำสุดในปี 2025" ต้องใช้ orders, order_items, products, categories และ reviews พร้อมกัน คนที่ไม่ถนัด SQL จึงต้องรอทีมข้อมูล

- **ผู้ใช้หลัก:** นักวิเคราะห์ธุรกิจ ผู้จัดการฝ่ายขาย/โลจิสติกส์ ที่รู้คำถามแต่ไม่รู้ schema
- **สิ่งที่ได้รับ:** คำตอบเป็นตัวเลขหรือตาราง พร้อม SQL ที่ตรวจสอบย้อนได้ และระบุว่าใช้ตารางใด
- **นอกขอบเขต:** การเขียน/แก้ข้อมูล, การพยากรณ์, dashboard แบบเต็มรูป

ข้อมูลนี้ไม่ใช่เนื้อหาของวิชา reference app (kmitl-llm-app-demo) จึงตอบคำถามเหล่านี้ไม่ได้ ตรงตามเงื่อนไขข้อ 1 ของโจทย์

ข้อมูลสังเคราะห์และ schema

ใช้ข้อมูลสังเคราะห์ของร้านค้าออนไลน์ไทยสมมติ ช่วงมกราคม 2024 ถึงธันวาคม 2025 รวม 8 ตาราง ประมาณ 50,000 แถว สร้างด้วย `scripts/generate_data.py` (Python stdlib ล้วน, `random.Random(42)`, `--seed 42`) แล้วโหลดเข้า SQLite ไฟล์เดียว รัน script ซ้ำต้องได้ข้อมูลเหมือนเดิมทุก byte ผลประเมินจึงทำซ้ำได้ ไม่ใช้ LLM สร้างแถวข้อมูล

ตาราง	เนื้อหา	คีย์สำหรับ join	แถว (เป้าหมาย)
order_items	สินค้าในคำสั่งซื้อ จำนวน ราคาต่อหน่วย ส่วนลด	order_id, product_id	~22,000
orders	สถานะ ช่องทาง คูปอง และ timestamp สร้าง/ชำระ/จัดส่ง/ถึงมือ	order_id, customer_id, coupon_id	~10,000
payments	วิธีชำระ (PromptPay, บัตร, โอน, COD) ยอดเงิน สถานะ	payment_id, order_id	~10,000
reviews	คะแนน 1–5 และความเห็นภาษาไทย	review_id, order_id	~4,000
customers	ชื่อ จังหวัด วันสมัคร สถานะสมาชิก อีเมล/เบอร์	customer_id	~3,000
products	ชื่อสินค้าไทย หมวด ราคา ต้นทุน	product_id, category_id	~500
coupons	รหัส ประเภทส่วนลด มูลค่า ช่วงใช้ได้	coupon_id	~30
categories	ชื่อหมวดไทยและอังกฤษ	category_id	~20

จงใจใส่ข้อมูลไม่สะอาดในสัดส่วนที่กำหนดได้ใน generator และเขียนกฎแต่ละข้อลง schema docs เพื่อวัดว่า retrieval ช่วยให้ agent กรองข้อมูลถูกจริงหรือไม่:

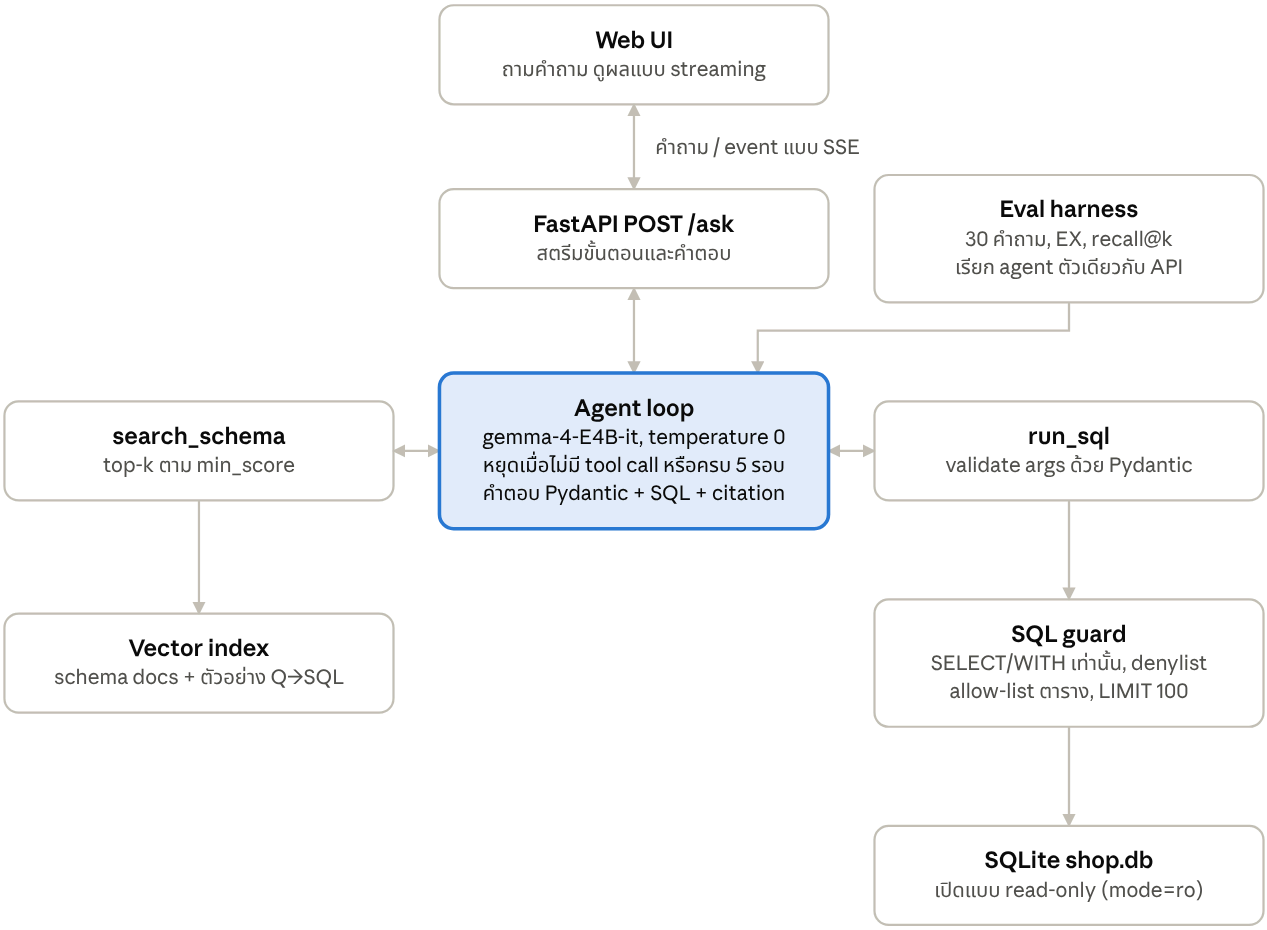
- **สถานะยกเลิกและคืนเงิน:** ยอดขายต้องไม่นับคำสั่งซื้อ `cancelled` และ `refunded`
- **ค่า NULL:** คำสั่งซื้อที่ยังไม่ส่งหรือถูกยกเลิกไม่มี `shipped_at` / `delivered_at`; สินค้าบางชิ้นไม่มีหมวด
- **ลูกค้าซ้ำ:** ราว 3% มีหลาย `customer_id` แต่อีเมลหรือเบอร์เดียวกัน คำถาม "ลูกค้ากี่คน" ต้องนับตามอีเมล
- **ชื่อจังหวัดไม่สม่ำเสมอ:** เช่น กรุงเทพ, กทม., กรุงเทพมหานคร, Bangkok หมายถึงที่เดียวกัน
- **การชำระเงินหลายรายการ:** บางคำสั่งซื้อแบ่งจ่ายหรือมีรายการชำระล้มเหลวก่อนสำเร็จ

ความเสี่ยงของข้อมูลสังเคราะห์คือ pattern เรียบเกินไปจนคำตอบไม่มีความหมาย generator จึงใส่แนวโน้มที่ตรวจได้ เช่น ยอดพุ่งช่วง 11.11 และ 12.12 และให้ความนิยมของหมวดต่างกันตามจังหวัด

สถาปัตยกรรมและ flow

ทั้ง API และ eval harness เรียก agent ตัวเดียวกัน ตัวเลขในรายงานจึงตรงกับพฤติกรรมของแอปที่สาธิต

SQL ทุกคำสั่งผ่าน guard ก่อนถึงฐานข้อมูลแบบ read-only



สถาปัตยกรรมระบบ · agent, tools 2 ตัว, eval harness

agent ดึงคำอธิบายตารางและตัวอย่างที่คล้ายผ่าน `search_schema` ก่อน แล้วส่ง SQL ผ่าน `run_sql` ซึ่งต้องผ่าน guard ทุกครั้ง ถ้ารันแล้ว error ข้อความ error จะกลับไปให้ agent แก้ในรอบถัดไป

Capability families ที่ใช้และเหตุผล

ครอบคลุม 5 จาก 6 families ตามโจทย์ ซึ่งรวม evaluation และ agentic สองตัว (เกินขั้นต่ำ 3)

Family	ใช้ทำอะไรในระบบ	ทำไมเลือก
(2) RAG + citation	embed คำอธิบายตาราง/คอลัมน์ และคลังตัวอย่าง Q→SQL แล้วดึง top-k ตาม min_score มาใส่ prompt; คำตอบอ้างชื่อตารางและ id ตัวอย่างที่ใช้	model ขนาดเล็ก (gemma-4-E4B-it) ไม่ควรรับ schema ทั้งหมด ตัวอย่างที่คล้ายช่วยเรื่อง join และ date filter

Family	ใช้ทำอะไรในระบบ	ทำไมเลือก
(3) Retrieval evaluation	ชุดทดสอบ ~30 คำถาม + gold SQL; วัด execution accuracy และ schema recall@k	เงื่อนไขบังคับ และเป็นตัวเลือก top-k/min_score ด้วยหลักฐาน
(4) Agent loop + tools	tools `search_schema`, `run_sql`; หยุดเมื่อ model ไม่เรียก tool หรือครบ 5 รอบ	ให้ agent ดู error แล้วแก้ SQL เอง ซึ่ง single-shot ทำไม่ได้
(5) Text-to-SQL	สร้าง SELECT แล้วรันบน SQLite read-only ผ่าน guard	แกนหลักของโปรเจกต์
(6) Served interface	FastAPI `POST /ask` สตรีมขั้นตอนของ agent และคำตอบด้วย SSE ไปยังหน้าเว็บง่ายๆ	ใช้สาธิตสด และให้ผู้ใช้เห็น SQL ระหว่างทำงาน

แม้ไม่นับเป็น family หลัก แต่จะใช้ structured output (Pydantic) กับ argument ของ tool และคำตอบสุดท้าย เพราะ rubric ของ repo ให้คะแนนเรื่อง validated structured output และ allow-listed tool calls

ความปลอดภัยของ SQL และ agent

ถือว่า SQL และ tool call ทุกอย่างจาก model เป็น input ที่ไม่น่าเชื่อถือ และป้องกันหลายชั้นตาม Lecture 9–11

1. **Read-only connection:** เปิด SQLite ด้วย URI `mode=ro` เป็นชั้นที่แข็งแรงที่สุด แม้ guard พลาดก็เขียนข้อมูลไม่ได้
2. **Guard:** ล้าง code fence และ `;` ท้าย, อนุญาตเฉพาะขึ้นต้นด้วย `SELECT` หรือ `WITH`, ปฏิเสธหลาย statement, denylist คำสั่ง เช่น `INSERT` `UPDATE` `DELETE` `DROP` `ATTACH` `PRAGMA`
3. **Allow-list ตาราง:** อนุญาตเฉพาะ 8 ตารางที่ใช้ ตรวจด้วย parser (เช่น `sqlglot`) ไม่ใช่ regex อย่างเดียว
4. **Limit และ timeout:** เติม `LIMIT 100` ถ้าไม่มี, ตั้ง timeout ต่อ query, ส่งผลกลับให้ model ไม่เกินจำนวนแถวที่กำหนด
5. **Tool allow-list:** รับเฉพาะ `search_schema` และ `run_sql`; validate argument ด้วย Pydantic ก่อนรัน, ชื่อ tool อื่นตอบกลับเป็น error
6. **Stop condition:** `max_steps = 5` และนับ token/การเรียกต่อคำถาม
7. **Config:** อ่าน `OPENAI_API_KEY` และ `OPENAI_BASE_URL` จาก environment, ไม่ commit `.env`, ใช้ `temperature=0` ทุกจุดที่ประเมินผล

แผนการประเมินผล

ตัวชี้วัดหลักคือ execution accuracy (EX) บนชุดทดสอบ 30 คำถาม ระบบต้องชนะ baseline B0 และต้องมีตัวเลขก่อน–หลังที่ใช้เลือก top-k/min_score

ชุดทดสอบ (`eval/testset.jsonl`): เขียนเอง 30 คำถาม แต่ละข้อมี gold SQL, ตารางที่ต้องใช้ และระดับความยาก แบ่งเป็นง่าย (1 ตาราง ไม่มีกับดัก) 10 ข้อ, กลาง (join 2–3 ตาราง หรือ 1 ตารางที่มีกับดักข้อมูล) 12 ข้อ, ยาก (join 4+ ตาราง หรือ aggregation ตามช่วงเวลา) 8 ข้อ รัน gold SQL ทุกข้อเพื่อยืนยันว่าได้ผลไม่ว่าง คำถามเป็นภาษาผสม ไทย 15 ข้อและอังกฤษ 15 ข้อ กระจายทุกระดับความยาก และรายงาน EX แยกตามภาษา เพราะ model เล็กอาจตีความคำถามภาษาไทยพลาดมากกว่า คลังตัวอย่าง few-shot ก็จะมีทั้งสองภาษาเช่นกัน

กันข้อมูลรั่ว: คลังตัวอย่าง few-shot (`data/examples.jsonl`) เขียนแยกจากชุดทดสอบ และไม่มีคำถามซ้ำหรือแค่เปลี่ยนตัวเลข

ตัวชี้วัด

- **Execution accuracy (EX):** ผลลัพธ์ของ SQL ที่สร้างตรงกับ gold แบบไม่สนลำดับแถว ปัดทศนิยม 2 ตำแหน่ง และยอมให้คอลัมน์เกินหรือสลับลำดับได้ พร้อมรายงาน EX-strict ที่คอลัมน์ต้องตรงทุกตัวควบคู่กัน
- **Valid SQL rate:** สัดส่วนที่ผ่าน guard และรันไม่ error
- **Schema recall@k:** สัดส่วนตารางที่ต้องใช้ซึ่งอยู่ใน top-k ที่ดึงมา
- **ต้นทุน:** จำนวนรอบ agent และ latency เฉลี่ยต่อคำถาม

$$\text{EX} = \frac{1}{N}\sum_{i=1}^{N} \mathbb{1}\left[\operatorname{exec}(\hat{q}_i) = \operatorname{exec}(q_i^{\ast})\right]$$

การทดลอง (`uv run --env-file .env eval/run.py --config <name>` , ผลลง `eval/results/`)

Config	Schema ใน prompt	Few-shot	Agent แก้ error	EX	Valid SQL
B0: baseline	ทั้งหมด	ไม่มี	ไม่มี (ยิงครั้งเดียว)		
B1: + retrieval	top-k ตาราง	ไม่มี	ไม่มี		
B2: + few-shot	top-k ตาราง	top-k ตัวอย่าง	ไม่มี		
Full system	top-k ตาราง	top-k ตัวอย่าง	สูงสุด 5 รอบ		

การปรับค่าที่วัดผล (measured improvement): ไล่ k ของตัวอย่างที่ 0, 1, 3, 5 และ min_score หลายระดับ วัดทั้ง EX และ recall@k แล้วเลือกค่าที่ใช้จริงจากตัวเลข ไม่ใช่การเดา

วิเคราะห์ความผิดพลาด: จัดกลุ่มข้อที่ผิด เช่น join ผิด, ใช้ timestamp ผิด, ลืมตัดคำสั่งซื้อที่ยกเลิก/คืนเงิน, นับลูกค้าซ้ำ, ชื่อจังหวัดไม่ตรงกัน แล้วเลือกตัวอย่างไปแสดงในรายงานและวิดีโอ

โครงสร้าง repo และแผนงาน

แยกชั้นตามแนวของวิชา: config → data → retrieval → agent/tools → API → UI โดยให้ eval เรียกชั้น agent ตัวเดียวกับที่ API ใช้

```
thai-shop-sql-agent/
├── pyproject.toml       # dependencies + entry points (จัดการด้วย uv)
├── uv.lock              # commit ไว้ ให้ทุกเครื่องได้เวอร์ชันเดียวกัน
├── .python-version      # เวอร์ชัน Python ที่ uv ใช้
├── README.md            # ทำอะไร, ติดตั้ง, รัน, รัน eval
├── .env.example         # OPENAI_API_KEY, OPENAI_BASE_URL (ไม่มีค่าจริง)
├── .gitignore           # .env, .venv/, data/*.db
├── scripts/
│   ├── generate_data.py # stdlib random, --seed 42 → CSV
│   └── build_db.py      # CSV → shop.db
├── data/
│   ├── schema_docs.md   # กฎธุรกิจ + คำอธิบายตาราง (1 chunk ต่อหัวข้อ)
│   └── examples.jsonl   # คลัง few-shot Q→SQL (แยกจาก test set)
├── src/thai_shop_agent/
│   ├── __init__.py
│   ├── config.py        # อ่าน env, model, temperature
│   ├── llm.py           # OpenAI client ผ่าน class proxy
│   ├── retrieval.py     # embed, index, search(min_score, k)
│   ├── sql_guard.py     # clean, guard, allow-list, add_limit
│   ├── tools.py         # search_schema, run_sql + Pydantic args
│   ├── agent.py         # loop, max_steps, citations
│   ├── schemas.py       # Pydantic models ของ tool และคำตอบ
│   └── api.py           # FastAPI POST /ask (SSE)
├── web/index.html       # UI ง่ายๆ รับ SSE
├── eval/
│   ├── testset.jsonl
│   ├── run.py           # รัน configs B0–Full
│   └── results/
└── tests/               # pytest: guard และ tools
```

คำสั่งหลักที่จะเขียนใน README (ใช้ `uv run --env-file .env` แบบเดียวกับที่สอนใน Lecture 1):

```
uv sync                                                    # สร้าง .venv จาก
uv.lock
uv run scripts/generate_data.py --seed 42 && uv run scripts/build_db.py
uv run --env-file .env uvicorn thai_shop_agent.api:app --reload
uv run --env-file .env eval/run.py --config full
uv run pytest
```

เริ่มโปรเจกต์ด้วย `uv init --package` แล้ว `uv add openai fastapi uvicorn pydantic` และ `uv add --dev pytest` ใช้โครงแบบ `src/` เพื่อให้ eval, tests และ API import package ตัวเดียวกัน

ลำดับงาน (ทำ eval ก่อนตามคำแนะนำในโจทย์):

1. ส่ง proposal และยืนยันขอบเขตกับอาจารย์
2. เขียน generator และโหลดข้อมูลเข้า SQLite, เขียน schema docs, เขียนชุดทดสอบ 30 ข้อและคลังตัวอย่าง
3. ทำ baseline B0 และ eval harness ให้ได้ตัวเลขแรก
4. ทำ retrieval, guard, agent loop แล้ววัด B1, B2, Full
5. ไล่ k/min_score, วิเคราะห์ความผิดพลาด
6. ทำ FastAPI + SSE UI, README, unit test
7. เขียนรายงาน 4–8 หน้า และอัดวิดีโอ 3–6 นาที (กรณีสำเร็จ 1 + ล้มเหลว 1)

ความเสี่ยง ข้อจำกัด และคำถามที่ยังเปิดอยู่

ความเสี่ยง	ผลกระทบ	ทางรับมือ
gemma-4-E4B-it เป็น model เล็ก อาจพลาด join 4+ ตาราง	EX ของข้อยากต่ำ	ลดขนาด schema ด้วย retrieval, เพิ่ม few-shot, รายงานแยกตามระดับความยากอย่างตรงไปตรงมา
model เรียก tool ผิดรูปแบบ หรือไม่หยุด	agent ค้าง/คำตอบเสีย	validate ด้วย Pydantic, ส่ง error กลับให้แก้, max_steps = 5
คำถามกำกวมเรื่องเวลา หรือนิยาม "ยอดขาย"	gold SQL ไม่ตรงความตั้งใจ	เขียนนิยามมาตรฐานใน schema docs (เช่น ยอดขาย = sum(จำนวน × ราคาต่อหน่วย − ส่วนลด) ของคำสั่งซื้อที่ไม่ถูกยกเลิก)
Embedding ผ่าน proxy อาจไม่มีให้ใช้	retrieval ใช้ไม่ได้	ตรวจกับ proxy ก่อน; สำรองด้วย embedding model local หรือ BM25

คำถามที่ยังเปิดอยู่

- [ ] ชุดทดสอบเป็นภาษาไทย อังกฤษ หรือผสม → ตัดสินแล้ว: ผสม
- [ ] Proxy ของวิชามี embedding model ให้ใช้หรือไม่
- [ ] วันส่ง proposal, รายงาน และวันสาธิตสด
- [ ] ยืนยันกับอาจารย์ตอนส่ง proposal ว่าข้อมูลสังเคราะห์ที่สร้างจาก script ผ่านเงื่อนไข "own domain" ของโจทย์